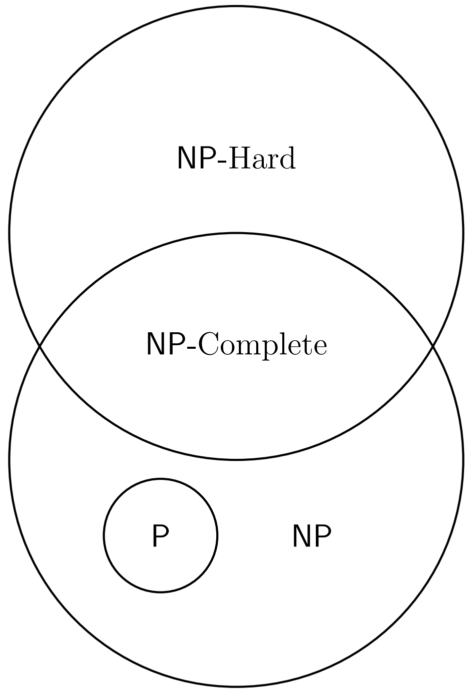
CS 3510: Design & Analysis of Algorithms 04/11/2023

Practice Exam 4: Intro to Complexity

Abrahim Ladha

- This is the CS 3510 practice exam for exam 4.
- Topics include: Algorithmic Complexity
- The number of questions on this practice exam is roughly what you could expect on the real exam.



1. For certain, this graph shows an accurate depiction of the relationship between the sets P, NP, NP-Hard, and NP-Complete (this is a trick question).
   - ◯ True
   - ◯ False
   - ◯ Maybe

2. If a quantum researcher discovers a way to run infinitely many polynomial-time algorithms at one time, then for quantum computers
   - ◯ They have proven P = NP
   - ◯ They have not proven P = NP

3. You've just made a break through and found a polynomial solution to the `LimitSAT`! The issue is that it's not exactly the same as normal `SAT` because each variable can only appear in at most three clauses, and there can only be at most three literals per clauses. Can you show that this is NP-Complete, thus showing $\mathsf{P} = \mathsf{NP}$ (Obviously this is a made up scenario because we don't actually know if $\mathsf{P} = \mathsf{NP}$, but nonetheless, show that `LimitSAT` is in NP-Complete)?

4. As a student, you're trying to determine your schedule for the upcoming semester. For a particular day, there's some set of classes $s \in S$, each class has some meeting times (a class can meet multiple times in the day i.e. lecture and recitation). You need to determine whether you can schedule k non-overlapping classes throughout the day. Show this problem is NP-Complete.

5. In `Daniel's-Two-Sum` problem, you're given a set of integers S, and you attempt to split the set up into two non-overlapping sets A and B such that $A \cup B = S$, $A \cap B = \emptyset$ and

$$\sum_{a \in A} a = \sum_{b \in B} b$$

Decide whether `Daniel's-Two-Sum` is NP-Complete and give sufficient evidence including either a proof it's NP-Complete or a polynomial time algorithm to solve it.

6. Show that the `Subgraph-Isomorphism` is in the class NP-complete. The problem is as follows: given as input two undirected graphs G and H, determine whether G is a subgraph of H.

7. The Frog Software Foundation has recently come out with Knapsack as a service, allowing you to solve the Knapsack decision problem, wherein you input your knapsack problem and a value, and the service will tell you if the best value in Knapsack is above the provided value. How could you use this to efficiently solve the Knapsack optimization problem wherein you're given the Knapsack parameters and you want to find the best possible value. Then discuss your algorithms runtime.